ACH2044 - Sistemas Operacionais

Exercício programa 2
2025

Objetivos **2**
Especificação da interface **3**
Algoritmo de compactação **4**
Algoritmo de descompactação **6**
Exigências da implementação **7**
Dicas **8**
Entrega **9**
Correção **10**

Objetivos

Para este exercício, você deverá fornecer uma implementação paralela de um algoritmo de compactação e descompactação de arquivos texto, que batizaremos de **EACHrink**.

Há três objetivos principais para esta exercício programa:

- Familiarizar-se com a biblioteca pthreads.
- Aprender a paralelizar um programa na prática.
- Aferir o impacto do número de threads no desempenho do programa.

Para implementar este exercício, você usará conceitos relacionados à concorrência, como criação de threads, exclusão mútua e sinalização. Estes tópicos foram tratados nas primeiras aulas sobre concorrência.

Especificação da interface

Você deve implementar um programa em linguagem C contido em um único arquivo com nome `eachrink<numero usp>.c` (por exemplo: `eachrink5894150.c`). O programa recebe quatro argumentos de linha de comando:

```
$ ./eachrink<nusp> <comando> <entrada> <saída> <threads>
```

O parâmetro <comando> deve ser o caractere `'c'` ou o caractere `'d'` para indicar se a operação realizada é de compactação ou descompactação.

O parâmetro `<entrada>` é o nome do arquivo de entrada que será processado pelo programa. No caso de ser operação de compactação, o arquivo deve conter apenas caracteres ASCII. No caso de ser operação de descompactação, o arquivo de entrada deve ser um arquivo previamente compactado.

O parâmetro `<saída>` é o nome do arquivo de saída que será produzido pelo programa.

O parâmetro `<threads>` deve ser usado para determinar a quantidade de threads adicionais que o programa usará (assumindo que a thread main não participa do processamento, realizando apenas a coordenação das outras threads). Se o parâmetro for igual a `0`, o programa deve decidir quantas threads usar.

Por exemplo, o comando abaixo irá compactar o arquivo livro.txt, salvando a versão compactada como livro.chrinked, utilizando 3 threads:

```
$ ./eachrink c livro.txt livro.chrinked 3
```

Algoritmo de compactação

Deverá ser implementada uma variação simplificada do algoritmo Byte-Pair Encoding (BPE). Como se trata de uma versão simplificada, **é obrigatório seguir a especificação abaixo.**

A ideia geral do algoritmo é simples: dada uma sequência de símbolos, deve-se encontrar o par de símbolos mais frequente e substituir as ocorrências deste par por um único símbolo de compactação, reduzindo o tamanho da sequência inicial. O procedimento é repetido até que a frequência do par de símbolos mais frequente seja um ou até que não haja mais símbolos de compactação disponíveis.

Por exemplo, vamos compactar o texto "`bananananica`":

1. Par de símbolos mais frequente: "an" (frequência 4)
2. Símbolo de compactação escolhido `*`
3. Sequência após a substituição: `b****ica`
4. Par de símbolos mais frequente: "`**`" (frequência 2)[1]
5. Símbolo de compactação escolhido `&`
6. Sequência após a substituição: `b&&ica`
7. Par de símbolos mais frequente: "b&" (frequência 1, empatado com outros pares)
8. Fim, sequência compactada final: `b&&ica`

Podemos resumir os passos da compactação com o pseudocódigo abaixo:

```
encontra par mais frequente
simbolo = SIMBOLO_INICIAL
enquanto frequência do par mais frequente > 1 e simbolo != SIMBOLO_FINAL:
      substitui ocorrências do par mais frequente por simbolo
      armazena associação (par -> simbolo) na memória
      simbolo = proximo_simbolo(simbolo)
      encontra par mais frequente
salva associações (par -> simbolo) no arquivo de saída
salva dados comprimidos no arquivo de saída
```

[1] Note que a frequência não é 3, como um algoritmo de varredura simples poderia calcular.

Para este exercício, vamos compactar apenas arquivos codificados em texto ASCII, ou seja, podemos considerar que os arquivos de entrada são compostos por bytes cujos valores estão entre 0 e 127 (ou 0x7F), incluindo estes valores.

Assim, vamos considerar que o valor de `SIMBOLO_INICIAL` deve ser 128 (ou 0x80) e que o valor de `SIMBOLO_FINAL` deve ser 255 (ou 0xFF). A função `proximo_simbolo`, portanto, simplesmente adiciona 1 ao valor passado como parâmetro.

As associações par-símbolo devem ser armazenadas de forma sequencial no arquivo de saída. Como o símbolo inicial é conhecido e constante, é possível omitir os símbolos e armazenar apenas os pares, mantendo-os em ordem. Para separar a tabela de pares-símbolos do texto compactado, deve-se introduzir um byte `SIMBOLO_FINAL` (0xFF) no arquivo de saída.

Por exemplo, vamos compactar novamente o texto “bananananica”, levando em conta estes requisitos:

1. Par de símbolos mais frequente: “an” (frequência 4)
2. Símbolo de compactação escolhido <0x80>
3. Sequência após a substituição: b<0x80><0x80><0x80><0x80>ica
4. Par de símbolos mais frequente: “<0x80><0x80>” (frequência 2)
5. Símbolo de compactação escolhido <0x81>
6. Sequência após a substituição: b<0x81><0x81>ica
7. Par de símbolos mais frequente: “b<0x81>” (frequência 1, empatado com os outros)
8. Fim, sequência compactada final: b<0x81><0x81>ica
9. Salva associações par-símbolo no arquivo de saída: an<0x80><0x80>
10. Adiciona um `SIMBOLO_FINAL (0xFF)` no arquivo de saída
11. Salva dados comprimidos no arquivo de saída: b<0x81><0x81>ica
12. Conteúdo final do arquivo comprimido:
    an<0x80><0x80><0xFF>b<0x81><0x81>ica

Note que a saída é composta por 11 bytes, enquanto que a entrada é composta por 12 bytes. Consequentemente, o processo de compressão reduziu um byte do arquivo de entrada.

Algoritmo de descompactação

O algoritmo de descompactação consiste, como era de se esperar, em desfazer tudo o que o algoritmo de compactação fez. Para isso, as associações par-símbolo devem ser recuperadas do arquivo compactado e as substituições devem ser desfeitas. A forma mais simples é seguir a ordem do símbolo de maior valor para o de menor valor (ou seja, a substituição feita por último na compactação deve ser desfeita primeiro na descompactação).

Vamos descompactar o arquivo compactado no exemplo anterior:

1. Arquivo de entrada: an<0x80><0x80><0xFF>b<0x81><0x81>ica
2. Associações par-símbolo: <0x80> -> an; <0x81> -> <0x80><0x80>
3. Sequência inicial: b<0x81><0x81>ica
4. Substituindo símbolo <0x81>: b<0x80><0x80><0x80><0x80>ica
5. Substituindo símbolo <0x80>: bananananica
6. Sequência final: bananananica
7. Escreve sequência final no arquivo de saída
8. Fim

Podemos resumir os passos da descompactação com o pseudocódigo abaixo:

```
símbolo = SIMBOLO_INICIAL

enquanto não leu SIMBOLO_FINAL:

      lê par de caracteres

      armazena associação (par -> símbolo) na memória

      símbolo = proximo_simbolo(símbolo)

Sequência inicial = bytes restantes

para cada símbolo armazenado, do maior para o menor:

      substitui ocorrências do símbolo pelo par associado

escreve arquivo de saída
```

Note que os pseudocódigos apresentados não necessariamente representam os algoritmos de compactação e descompactação mais eficientes.

Exigências da implementação

O seu programa deve obedecer às seguintes exigências:

1. Durante a compactação, no caso de empate do par com a maior frequência, escolha o par que vem antes numericamente (considerando a tabela ASCII para os valores dos caracteres de texto). Por exemplo:
   'AA' < 'aa' < 'ab' < 'ba' < 'b<0x80>' < '<0x80>a'.

2. O arquivo de entrada não deve ser modificado.

3. O resultado da compactação deve ser determinístico e independente da quantidade de threads utilizadas.

4. Você pode assumir que o arquivo de entrada é grande o suficiente em relação ao número de threads, de forma que o número de bytes do arquivo dividido pelo número de threads é maior ou igual a 1.

5. Você pode assumir que os arquivos de entrada e de saída cabem inteiros na memória ao mesmo tempo. Ou seja, não é necessário carregar e processar os arquivos por partes.

6. Você pode assumir que o arquivo de entrada possui menos de 2147483647 bytes (equivalente a 2GB - 1). Consequentemente, a frequência máxima de um certo par de caracteres será inferior a 2147483647, ou seja, o valor de frequência pode ser armazenado numa variável do tipo `int` sem risco de ocorrer overflow (considerando que as variáveis deste tipo possuem 32 bits).

7. Não é necessário escrever nada na saída padrão (prints), mas podem ser adicionados se desejado. É recomendável remover os prints para realizar análise de desempenho (explicada mais adiante).

Dicas

Seguem algumas dicas de coisas que você precisará pensar sobre e/ou resolver para implementar o exercício:

1. Como paralelizar o processo de compactação e descompactação? Este é o desafio central deste projeto. Pense no que pode ser feito em paralelo e no que deve ser feito em série por uma única thread e projete sua compactação e descompactação paralela conforme apropriado.

2. Como determinar quantas threads criar? No Linux, é possível usar funções como `get_nprocs()` e `get_nprocs_conf()` para determinar a quantidade de processadores disponíveis no sistema (e basear a quantidade de threads usadas no valor obtido). Leia as páginas do manual (com `man get_nprocs`) para saber mais detalhes.

3. Como executar cada parte do trabalho de forma eficiente? Embora a paralelização tenha o potencial de aumentar a velocidade, a eficiência de cada thread também é importante.

4. Como acessar os arquivos de entrada/saída de forma eficiente? No Linux, há muitas maneiras de ler um arquivo, incluindo funções da biblioteca padrão C como o `fread()` e funções de acesso de baixo nível como o `read()`. Uma maneira alternativa que pode ser eficiente é usar arquivos mapeados na memória, criados através da função `mmap()`. Ao mapear o arquivo de entrada no espaço de endereçamento do processo, você pode acessar os bytes do arquivo de entrada via ponteiro (como se fosse um vetor).

5. O programa hexdump pode ajudar a inspecionar os resultados produzidos pelo seu programa. Recomendo utilizá-lo com a flag -C.

Entrega

Além do código, que deve ser entregue como um único arquivo .c com nome `eachrink<numero usp>.c` (por exemplo, eachrink5894150.c), você deverá entregar um relatório nomeado como `<número usp>.pdf` (por exemplo, 5894150.pdf) contendo duas seções:

1) uma seção com as decisões de projeto que você tomou; e
2) uma seção com uma análise de desempenho do seu programa

As decisões de projeto incluem elementos tais como: as estruturas de dados utilizadas, como os algoritmos apresentados em pseudocódigo foram ajustados (ou se foram buscadas alternativas em outras fontes), estratégias de paralelização, definição das regiões críticas e como foram protegidas, além de qualquer outra informação que tenha sido relevante para a implementação.

A análise de desempenho deve relacionar o tempo de execução total do seu programa com o número de threads adicionais utilizadas (ou seja, variando o parâmetro `<threads>`). Varie a quantidade de threads adicionais pelo menos entre 1, 2, 4 e 8. Faça a análise para pelo menos dois tamanhos de arquivo de entrada diferentes (um “grande”, no qual o seu programa demore ao menos alguns segundos para executar, e um “pequeno”, no qual seu programa demore entre dezenas e centenas de milissegundos para executar).

Documente o método de medição utilizado e não esqueça de apresentar resultados com significância estatística, isto é, para cada par (número de threads, tamanho do arquivo de entrada) faça a medição de tempo várias vezes e calcule a média e o intervalo de confiança. É recomendado automatizar a execução (por exemplo, com um script).

Lembre-se de interpretar os resultados! Estavam de acordo com o esperado? Houve outliers?

Correção

Para os testes, seu código será ser compilado em ambiente Linux com as seguintes flags de compilação:

```
-Wall -Werror -pthread -std=c11 -O
```

Primeiramente, seu programa será testado quanto à corretude, ou seja, será verificado se ele é capaz de compactar arquivos e descompactá-los na sequência.

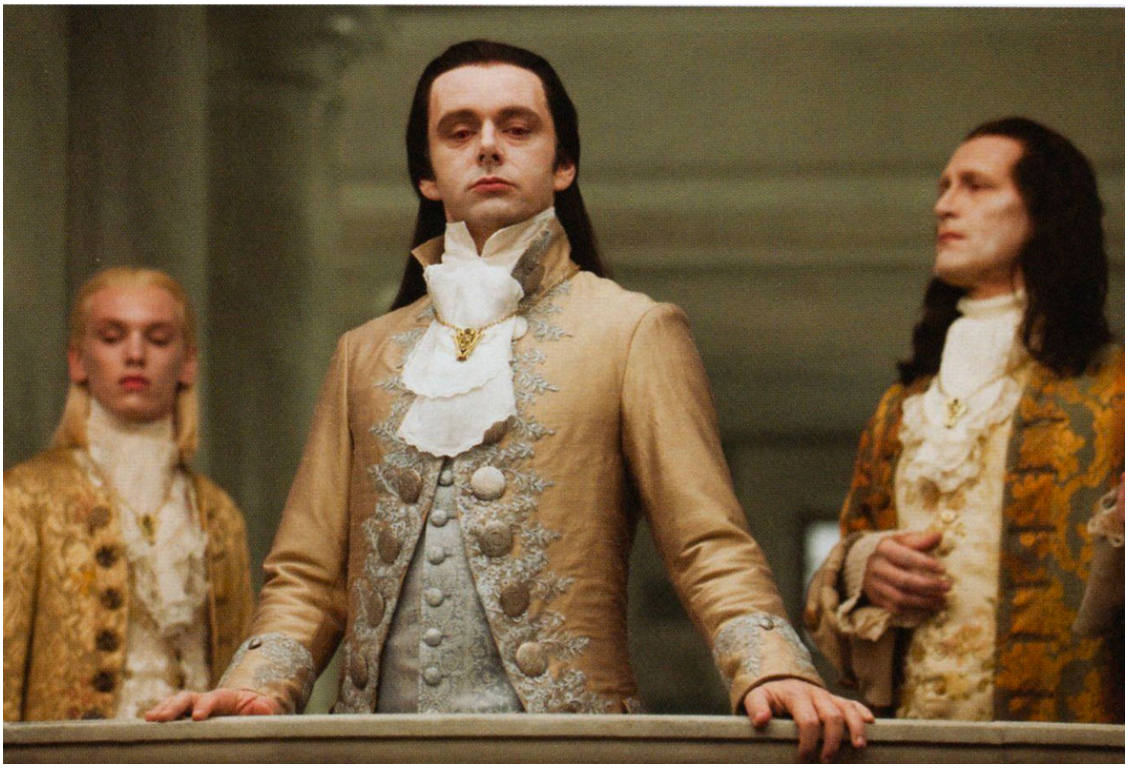
Em seguida, serão feitos testes de aderência à especificação, ou seja, se os arquivos compactados estão de acordo com o esperado e se as exigências foram atendidas.

Serão testados arquivos de três tamanhos diferentes (aproximadamente 1KB, 100KB e 1MB). Exemplos de arquivo de entrada e as respectivas saídas esperadas estão disponíveis no eDisciplinas.

Se o seu programa passar em todos os testes, seu código será testado quanto ao desempenho utilizando o comando `time`. Será avaliado o tempo total de execução para comprimir um arquivo grande (maior que os usados nos testes de corretude), passando-se `0` como número de threads.

O programa mais rápido será proclamado o "**compactador mais rápido de todos da sala**"!

E você poderá olhar assim para seus colegas:



Como de costume, os arquivos submetidos serão verificados por plágio, acarretando nota zero se for constatado. É permitido aos alunos discutirem estratégias de resolução, de preferência após pensar no problema por si, porém a implementação deve ser feita individualmente[2].

[2] Afinal, encontrar bugs em programas paralelos é parte do aprendizado :)